601.433/633 Introduction to Algorithms **Fall 2026**
Homework #7 **Due:** November 5, 2026, 11:59pm

Remember: this is an oral presentation homework! You are required to work in groups of three, and may collaborate only with other students in your group. We encourage you to write up solutions, but your grade will be based entirely on your presentation. At your presentation, the grader will randomly assign each student to present a problem. You should be able to create a group and sign up for a slot using Canvas. The deadline for signing up for a presentation slot is 5pm Wed Nov 4.

You may use the internet or LLMs to look up formulas, definitions, concepts, etc., but may not simply look up the answers online or ask an LLM.

1 Faster Shortest Paths

Let $G = (V, E)$ be a directed graph with lengths $\ell : E \to \mathbb{R}$ with no negative-length cycles. Let $v \in V$, and let α denote the maximum, over all $u \in V$, of the number of edges on a shortest path from v to u (where the shortest-path is defined with respect to the weights). Given G, w, and v (but not α), give an algorithm that computes shortest path distances from v to all other nodes in $O(m\alpha)$ time. You may assume that $m = \Omega(n)$. Prove correctness and running time.

Algorithm Description:

We use a modified verision of the Bellman-Ford dynamic programming algorithm, and we incorporate a stopping time. We define $\delta(u)$ for the shortest-path distance from v to u ($\delta(u) = \infty$ if u is unreachable). Let α be the maximum over reachable u of the minimum number of edges on a shortest v to u path, so every reachable u has a shortest path with at most α edges.

Subproblem. Let $\mathrm{OPT}(u, i)$ be the minimum length of a path from the start vertex v to u which uses at most i edges. We want to find a large enough i, for which $\mathrm{OPT}(u, i)$ finds the shortest path distance $d(v, u)$ for all u.

Recurrence.

$$\mathrm{OPT}(u,i) = \begin{cases} 0 & \text{if } k = 0, u = v \\ \infty & \text{if } k = 0, u \neq v \\ \min(\mathrm{OPT}(u, i-1), \min\limits_{w:w,u \in E}(\mathrm{OPT}(w, i-1) + \ell(w,u))) & \text{otherwise.} \end{cases}$$

Pseudocode. We use the Bellman-Ford algorithm, except instead of running the outer loop $n - 1$ times, we implement a tracker of changes to the distance array, which allows for early termination on the first loop where no distances are updated (indicating that the shortest path from v to all u has been found). The outer loop runs at most $\alpha + 1$ times as per the termination constraint.

1: $\mathrm{M}(v, 0) \leftarrow 0$;
2: $\mathrm{M}(u, 0) \leftarrow \infty$ **for** all $u \neq v$;
3: $i \leftarrow 0$;
4: **repeat**;

5: i ← i + 1;
6: **for do**$u \in V$ $\mathrm{M}(u, i) \leftarrow \min(\mathrm{M}(u, i-1), \min_{w:w,u \in E}(\mathrm{OPT}(w, i-1) + \ell(w, u)))$;
7: **end for**
8: **until** M(u, i) = M(u, i-1) **for** all $u \in V$; **return** M(., i);

Correctness Proof:

Claim 1 (Upper bound is never violated) *At all times, $d[u] \geq \delta(u)$ for all u.*

Proof: We perform induction on the number of relaxation steps performed so far.

Base case. (no relaxations yet): $d[v] = 0$ and $\delta(v) = 0$, so $d[v] \geq \delta(v)$. Every other $d[u] = \infty \geq \delta(u)$.

Inductive step. Assume $d[u] \geq \delta(u)$ for all u before some relaxation step on an edge (x, y). Only $d[y]$ can change, so it suffices to check y. If $d[y]$ does not change, the inequality still holds by the hypothesis. If $d[y]$ changes, the new value is $d[x] + \ell(x, y)$, and it is finite, so $d[x]$ is finite. Then

$$d[y]_{\text{new}} = d[x] + \ell(x, y) \geq \delta(x) + \ell(x, y) \geq \delta(y)$$

The first inequality is the inductive hypothesis applied to x. The second is the triangle inequality $\delta(y) \leq \delta(x) + \ell(x, y)$: a shortest path to x followed by the edge (x, y) is a walk to y, and since G has no negative cycles, any walk from v to y has length at least $\delta(y)$ (deleting cycles from the walk never increases its length and leaves a simple path). So the inequality holds after the step, completing the induction. ■

Claim 2 *After round i, for every u having a shortest path from v with at most i edges, $d[u] = \delta(u)$.*

Proof: We perform induction on i.

Base case. $i = 0$: only $u = v$, and $d[v] = 0 = \delta(v)$.

Inductive step. Suppose the claim holds after round $i - 1$, and let u have a shortest path $v = x_0, x_1, \ldots, x_k = u$ with $k \leq i$ edges. Then $x_0, \ldots, x_{k-1}$ is a shortest path to x_{k-1} with $k-1 \leq i-1$ edges (subpaths of shortest paths are shortest), so $d[x_{k-1}] = \delta(x_{k-1})$ after round $i-1$. In round i the edge (x_{k-1}, u) is relaxed, giving $d[u] \leq d[x_{k-1}] + \ell(x_{k-1}, u) \leq \delta(x_{k-1}) + \ell(x_{k-1}, u) = \delta(u)$. Combined with Claim 1, $d[u] = \delta(u)$. ■

Termination: By Claim 2, after round α we have $d[u] = \delta(u)$ for every reachable u; unreachable vertices keep $d[u] = \infty = \delta(u)$ throughout (an edge from a vertex with $d = \infty$ never triggers a relaxation, and unreachable vertices have no reachable in-neighbors).

In round $\alpha + 1$, every edge (x, y) satisfies $d[x] + \ell(x, y) \geq d[y]$ because $d = \delta$. Therefore, no change occurs, so $M(u, \alpha+1) = M(u, \alpha)$ for all u, making the algorithm return after $\alpha+1$ rounds.

Output. The algorithm only stops after a round with no change, but by the above it must stop by round $\alpha + 1$, at which point $d = \delta$. Hence the output is correct.

Runtime Proof:

Initializing $d[u]$ for all $u \in V$ takes $O(n)$. Since $m = \Omega(n)$, this step takes $O(m)$.

For each u, we look at every incoming edge (w, u) once, so over all u, we look at every edge once. This takes $O(n + m)$. Checking whether $M(u, i) = M(u, i-1)$ for all u takes $O(n)$. Since $m = \Omega(n)$, one round takes $O(m)$.

By the termination proof, the outer loop runs at most $\alpha + 1 \leq 2\alpha$ times (when $\alpha \geq 1$).

Synthesizing these steps, we get the following upper-bound

$$O(m) + (\alpha + 1) \cdot O(m) = O(m) + O(m\alpha) = O(m\alpha)$$

2 Multi-Set Shortest Paths

Let $G = (V, E)$ be a directed graph with weighted edges; edge weights can be positive, negative, or zero. There are no negative-weight cycles. Suppose the vertices of G are partitioned into k disjoint subsets $V_1, V_2, \ldots, V_k$; that is, every vertex of G belongs to exactly one subset V_i. For each $i, j \in [k]$, let $\delta(i, j)$ denote the minimum shortest-path distance between vertices in V_i and vertices in V_j:

$$\delta(i, j) = \min \{d(v_i, v_j) \mid v_i \in V_i \text{ and } v_j \in V_j\}$$

Design an algorithm to compute $\delta(i, j)$ for all $i, j \in [k]$ that runs in time $O(mn + kn \log n)$. You may assume that $m = \Omega(n)$. Prove correctness and running time.

Algorithm Description:

We use a modified version of Johnson's algorithm.

Reweighting, as in Johnson's algorithm. We construct $G' = (V', E')$, where $V' = V \cup \{q\}$ for a new vertex q, and $E' = E \cup \{(q, x) : x \in V\}$ with $w(q, x) = 0$ for all $x \in V$. We run Bellman-Ford on G' from q, and let $h(x) = d(q, x)$ for all $x \in V$. Define the new weight $w'(u, v) = w(u, v) + h(u) - h(v)$ for all $(u, v) \in E$, and let d' denote the shortest path distance with respect to w'.

Super source for each group. From here, we can run multi source Dijkstra, but we only need to find the distance between two subsets, not the distance between every node. Therefore, we run Dijkstra's once from each subset instead.

We create a source vertex s_i for each set V_i and we set its distance from each vertex in V_i to $C_i - h(u)$, where C_i is the maximum potential in V_i, and $h(u)$ is the potential of any vertex u in V_i. This cancells out the reweighting of Johnson's algorithm, and introduces C_i as a means of normalization to keep all distances nonnegative.

Finding distance between groups. For each $i, j \in [k]$, return $\delta(i, j) = \min\limits_{y \in V_j}\{D_i(y) + h(y)\}$.

Correctness Proof:

q has no incoming edges, so G' has no negative-weight cycle and h is well defined. By the triangle inequality, $h(v) \leq h(u) + w(u, v)$, so $w' \geq 0$. For any path p from x to y, the sum telescopes to $w'(p) = w(p) + h(x) - h(y)$, so $d'(x, y) = d(x, y) + h(x) - h(y)$

Thus, for each set V_i, we want the algorithm to return:

$D_i(v) = \min_{u \in V_i}(d'(u, v)) = \min_{u \in V_i}(d(u, v) + h(u) - h(v))$

Since the starting distance between the source and u is $C_i - h(u)$, we must add that to this equation to correctly find the minimum:

$D_i(v) = \min_{u \in V_i}(C_i - h(u) + d(u, v) + h(u) - h(v)) = \min_{u \in V_i}(C_i + d(u, v) - h(v))$

Since C_i and $h(v)$ are constants in this context, we can rearrage:

$\min_{u \in V_i}(d(u, v)) = D_i(v) - C_i + h_v$

As $\delta(u, v) = \min_{u \in V_i}(d(u, v))$, we can see that this equation correctly finds the distance. Therefore, the changed source weights allow for accurate calculation of any $\delta(u, v)$.

Runtime Proof:

G' has $n+1$ vertices and $m+n$ edges, so Bellman-Ford takes $O((n+1)(m+n)) = O(mn)$, since $m = \Omega(n)$. Computing w' then takes $O(m)$.

Each G_i has $n+1$ vertices and at most $m+n$ edges, so Dijkstra's algorithm with a Fibonacci heap takes $O(m + n\log n)$. Over all i, this takes $O(km + kn\log n)$. Since $V_1, \ldots, V_k$ partition V, $k \leq n$, so $km \leq mn$.

For a fixed i, one scan over all $y \in V$ gives $\delta(i, j)$ for all j, in $O(n)$. Over all i, this takes $O(kn)$.

Therefore, the total running time is $O(mn) + O(mn + kn\log n) + O(kn) = O(mn + kn\log n)$.

3 Single-Source Shortest Paths

Let $G = (V, E)$ be a connected, undirected graph with positive edge lengths $\ell : E \to \mathbb{R}^+$. This graph represents a road network. You are trying to get from $s \in V$ to $t \in V$. Unfortunately, you are a coffee addict: getting caffeine often enough is the most important thing to you. In fact, there is some number $L \in \mathbb{R}^+$ so that if you go more that L distance without getting coffee, you will get a caffeine migraine and crash your car. So you simply *cannot* go more than L distance without stopping for coffee. Fortunately, there is a coffee shop at every vertex $v \in V$ (including s and t) where you can get coffee. However, you are also a coffee snob, and you *hate* Starbucks. And there is a special subset of nodes $S \subseteq V$ where the only coffee shop on that node is a Starbucks.

So you have the following problem: given a subset $S \subseteq V$ of vertices where the only coffee shop is a Starbucks, find the path from s to t which uses only edges of length at most L, and has the *fewest* vertices in S. You may assume that $s, t \notin S$. Design an $O(m + n\log n)$-time algorithm for this problem, and prove running time and correctness.

Algorithm Description:

We use Dijkstra's algorithm with a modified version of the inputted graph G.

Step 1: G' Construction. We modify G to create G'. We do so by iterating through the edges of G. For all edges (u, v) in G, we perform the following steps:

1. If $\ell(u, v) > L$, do not add (u, v) to G' and continue to the next edge.
2. Split (u, v) into two directed edges $(u \to v)$ and $(v \to u)$
3. We set each edge weight $(u \to v)$ and $(v \to u)$ as follows:

$$w(u \to v) = \begin{cases} 1 & \text{if } v \in S, \\ 0 & \text{if } v \notin S. \end{cases}$$

$$w(v \to u) = \begin{cases} 1 & \text{if } u \in S, \\ 0 & \text{if } u \notin S. \end{cases}$$

Step 1: G' Traversal. From here, we perform Dijkstra's algorithm on G' from s. If t is unreachable from s (i.e. $d(t) = \infty$), we report no valid path exists. Otherwise, we follow the parent pointers of t back to s to reconstruct the path.

Correctness Proof:

Claim 3 *A path P from s to t in G uses only edges of length at most L if and only if P is a path from s to t in G'.*

Proof: By definition, E' consists exactly of the edges of G of length at most L, and G' has the same vertex set as G. Hence a path in G uses only such edges if and only if every one of its edges lies in E', i.e. if and only if it is a path in G'. ■

Claim 4 *Let $P = (s = v_0, v_1, \ldots, v_k = t)$ be a path in G'. Then the total w-weight of P equals the number of vertices of P that lie in S.*

Proof: The weight of P is $\sum_{i=1}^{k} w(v_{i-1}, v_i) = \sum_{i=1}^{k} [v_i \in S]$, which counts the vertices among $v_1, \ldots, v_k$ that lie in S. Since $s = v_0 \notin S$, the vertex v_0 contributes nothing to the count of S-vertices on P, so this sum equals $|V(P) \cap S|$. ■

Theorem 5 *The algorithm outputs a path from s to t that uses only edges of length at most L and has the fewest vertices in S among all such paths. If no such path exists, the algorithm reports this.*

Proof: By Claim 3, the feasible paths (those using only edges of length at most L) are exactly the s to t paths in G'. If there are none, then t is unreachable from s in G', so Dijkstra's algorithm gives $d(t) = \infty$ and the algorithm correctly reports that no valid path exists.

Otherwise, all arc weights w are nonnegative, so Dijkstra's algorithm correctly computes a minimum-w- weight path P^* from s to t in G'. By Claim 4, the w-weight of any s–t path equals the number of its vertices in S. Hence P^* minimizes the number of S-vertices over all s–t paths in G', i.e. over all feasible paths. Finally, the path returned via parent pointers is a simple path, so it is a valid answer. ■

Runtime Proof:

We first iterate through each of the m edges in G and perform a constant time operation on those edges to assign weights. Thus, the creation of G' has time $O(m + n)$.

Then, we run Dijkstra's algorithm. With a Fibonacci heap, it runs in $O(m + n \log n)$ time (n `Extract-Min` operations at $O(\log n)$ amortized each, and at most $2m$ `Decrease-Key` operations at $O(1)$ amortized each).

Finally, recovering the path from the parent pointers takes $O(n)$ time.

This yields a total running time of:

$$O(m + n) + O(m) + O(m + n \log n) + O(n) = O(m + n \log n)$$